

Continual Learning & Memory Models

COMS 6998

Week #4: February 18, 2026

Paper title: MemoryLLM: Towards Self-Updatable Large Language Models

Motivation and Background

Author: Mateo Juliani

Problem

LLMs typically remain static once deployed, which can lead to a deterioration in performance over time

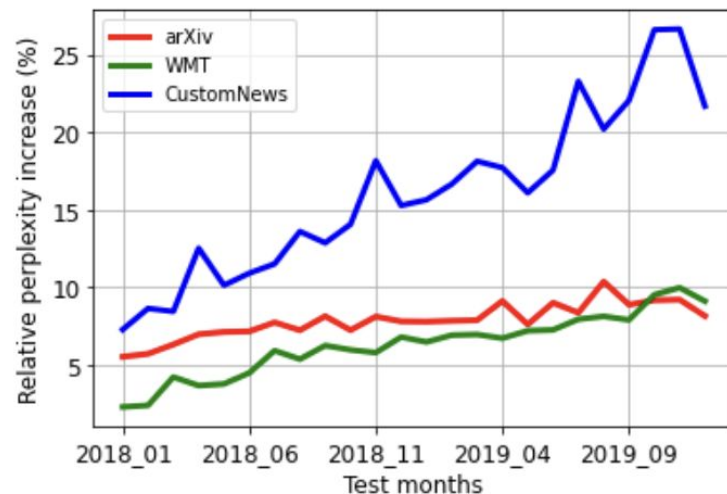


Figure 1: Relative ppl. increase of TIME-STRATIFIED over CONTROL, across test months.

Current Techniques

- Long Context Methods
 - Amount of new information limited by context lengths
- Retrieval-based Methods (RAG)
 - Can be redundant
- Model Editing Methods (KN, ROME)
 - Current methods are limited to small (1 sentence) updates at a time

Primary Question

Author: Mateo Juliani

Primary Question

How can we create an LLM that is capable of efficiently integrating new knowledge it encounters while minimizing the degradation of previously learned knowledge?

Author: Mateo Juliani

Primary Question

How can we create an LLM that is capable of efficiently integrating new knowledge it encounters while minimizing the degradation of previously learned knowledge?

Answer: MemoryLLM

Author: Mateo Juliani

Essential Properties

- Efficiency
 - Injecting new knowledge into the LLM should be fast
- Efficacy
 - Newly injected knowledge should have an impact on outputs
- Integrity
 - New knowledge updates should not reduce model performance
- Knowledge Retention
 - Old knowledge should be slowly phased out

Methods

Author: Ethan Simpson

ϕ

Static Parameters instantiated from
Llama2 layers

“Persistent Truths”

 θ

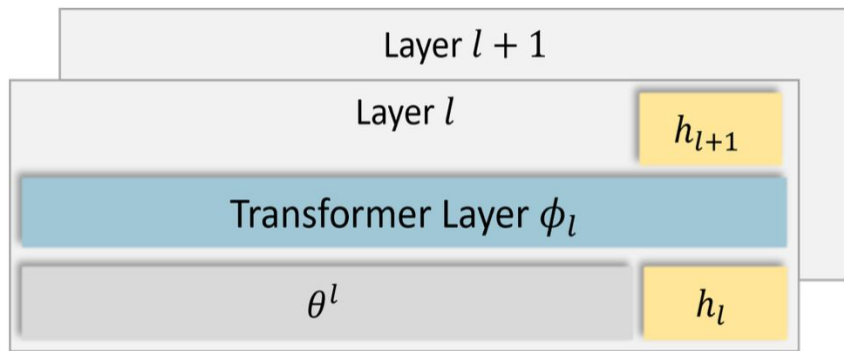
Dynamically Evolving Memory Pool

“Fresh Information”

$$\theta' = U(\theta, x)$$

Author: Ethan Simpson

Generation

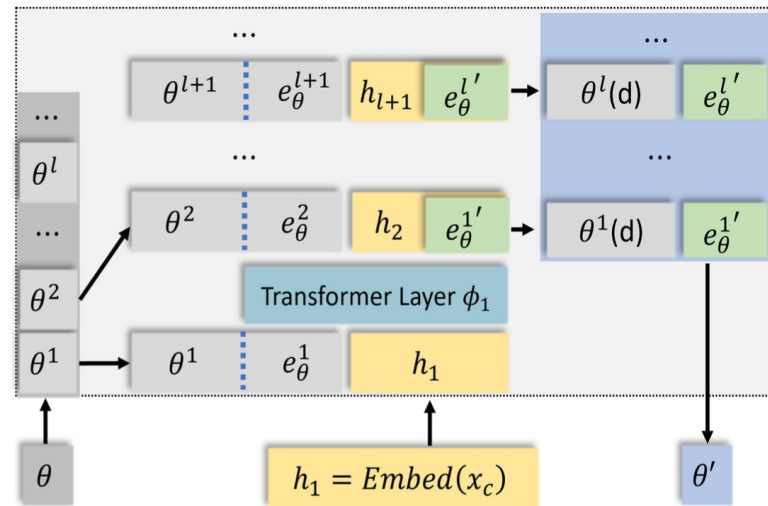


(a) Generation

Attention Map: $n_x \times (n_x + N)$

$$\phi = \{\phi_l\}_{l=1}^L \quad \theta = \{\theta_l\}_{l=1}^L$$

Self-Update

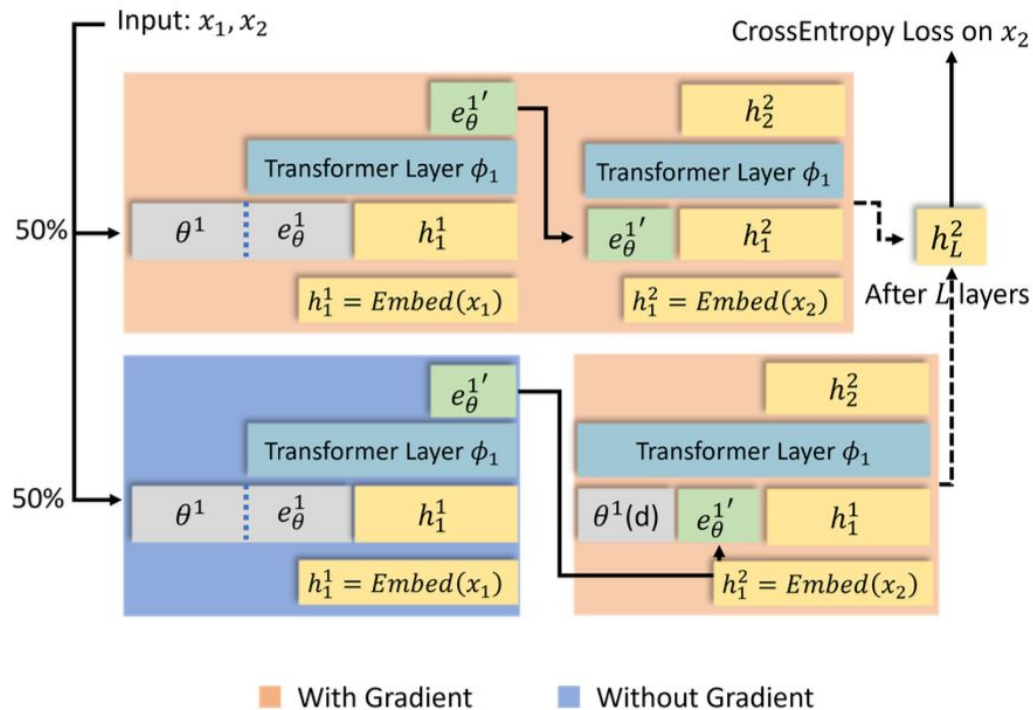


(b) Self-Update

Attention Map: $(n_{x_c}, K) \times (n_{x_c} + K)$

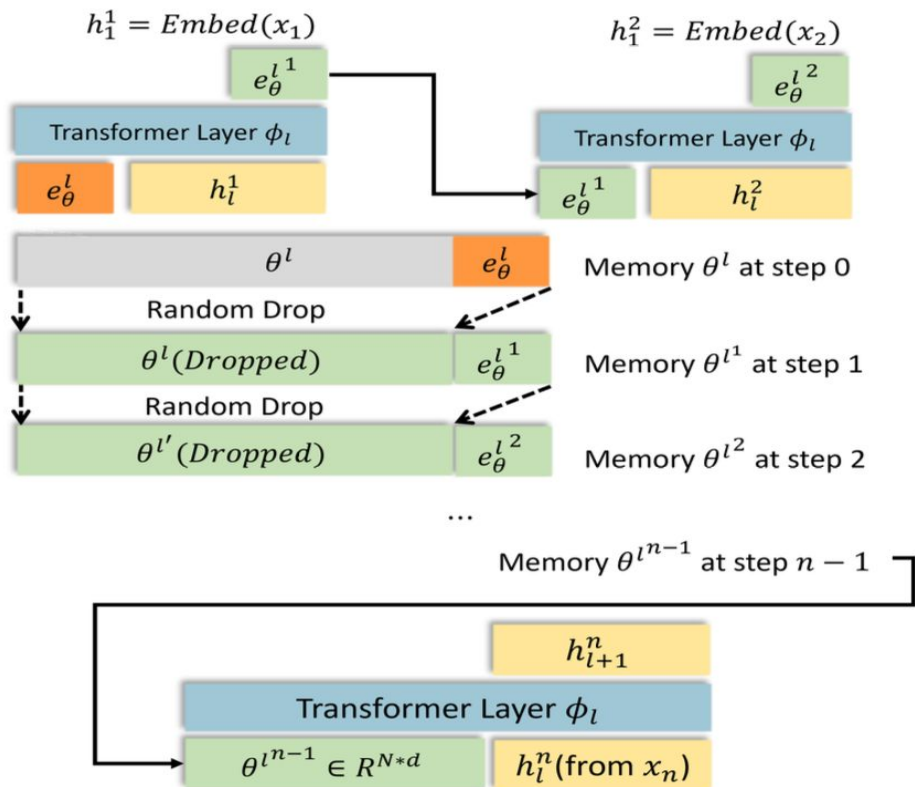
Author: Ethan Simpson

Training



Author: Ethan Simpson

What About Long Contexts?



Author: Ethan Simpson

Results: Metrics

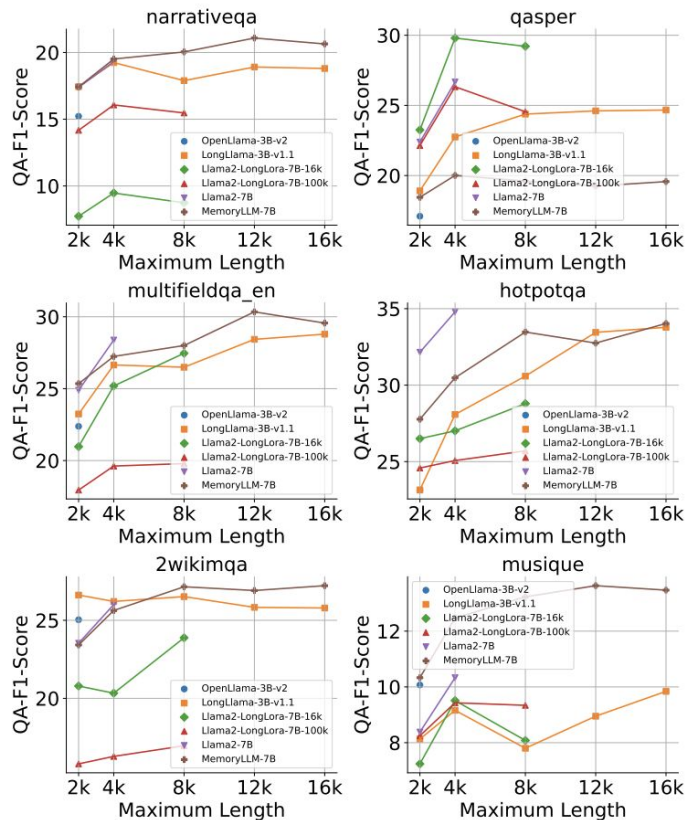
- Integration of New Knowledge
- Knowledge Retention Ability
- Robustness

Knowledge Integration: Quantitative Editing

Table 1. **Quantitative Editing Results on Llama2-7B** for ZsRE and CounterFactual Datasets. “w/EF” means “with editing facts”, indicating the model after updating the memory pool with the new fact.

Editor	ZsRE Dataset				CounterFactual Dataset			
	Score	Efficacy	Generalization	Specificity	Score	Efficacy	Generalization	Specificity
Llama2-7B	55.6	55.9	54.7	56.3	20.7	13.7	16.6	83.4
MemoryLLM-7B	51.2	50.0	49.1	54.8	22.6	15.7	17.6	82.1
FT	50.3	78.6	80.6	29.0	10.0	99.7	96.9	3.6
FT-L	69.8	81.4	76.8	56.6	33.8	47.2	18.0	83.3
ROME	69.3	88.7	70.2	56.3	69.2	82.6	75.2	55.8
IKE	-	-	-	-	70.7	99.8	96.2	45.4
MemoryLLM-7B (w/ EF)	79.2	99.8	96.7	57.1	75.3	98.5	82.2	57.0

- **Objective:** Assess the ability to learn specific new facts (Efficacy) without destroying existing knowledge (Specificity)



Knowledge Integration: Long Context Understanding

Performance:

- **Rising Curve:** Performance **improves** as context length increases
- **Exception:** Performance drops on scientific papers due to domain mismatch in Llama2 training

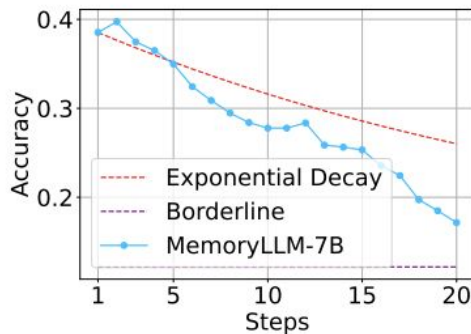
Author: Jeremiah Mao

vs RAG

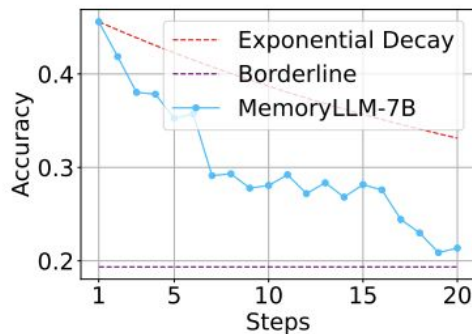
- **RAG is orthogonal to MemoryLLM**, hybrid approach showed mixed results

	MEMORYLLM-16k	MEMORYLLM-all-BM25
narrativeqa	20.64	15.60
qasper	19.57	20.30
multifieldqa_en	29.56	33.08
hotpotqa	34.03	32.27
2wikimqa	27.22	24.17
musique	13.47	15.36

Knowledge Retention & Robustness

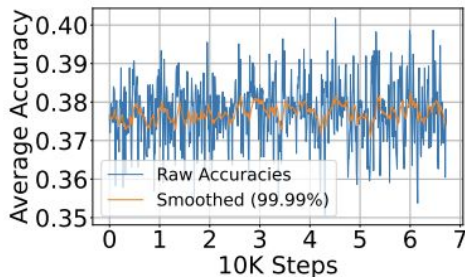


(a) SQuAD

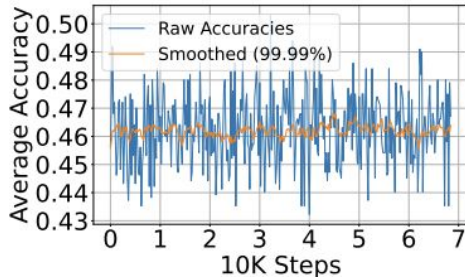


(b) NaturalQA

Retention: measures how long the model can retain specific information before it is overwritten



(a) SQuAD



(b) NaturalQA

Integrity: model continues to function correctly after an extreme number of updates

Author: Jeremiah Mao

Ablation Study

The N/K Ratio (Forgetting vs. Capacity)

- **Higher N/K is better:** Increasing the total memory (N) or decreasing the update tokens (K) significantly improves how long the model remembers facts.
- **A Lower Limit:** Reducing K to certain threshold makes the model "forget" almost instantly, with accuracy dropping by nearly 30%

Result Summary

- Exceptional Stability and Performance
- Tunable Forgetting (N/K ratio)
- Efficiency: It integrates new info instantly at inference
- avoid OOM errors from frontier models (only need 48 GB GPU)

Author: Jeremiah Mao

Core Contributions

- Introduces a fixed-size latent memory pool, θ , that can be self-updated at inference
- Uses transformer itself as a memory writer
- Replaces K tokens per update
- Maintains exponential forgetting with explicit half-life control (K/N)

Strengths

- Clean separation between static model, ϕ , and dynamic memory, θ
- Lightweight deployment
 - No gradient-based model editing
 - No retraining required at inference
- Explicit and tunable control of half-life via K/N
- Empirical demonstration of stability under many sequential updates

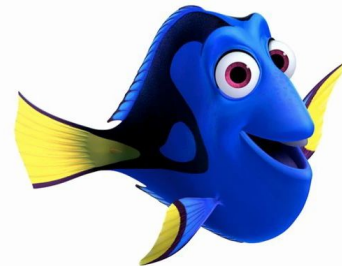
Weaknesses

- Fixed N memory tokens per layer
 - No mechanism for selective importance weighting
- Dropping policy
 - Could explore some learned eviction policy
 - Learn how to rank or judge importance of facts stored in dynamic memory
- Use of the same transformer unit for the dynamic memory
 - A dedicated writer network may improve compression quality at cost of complexity
- Comparison to RAG restricted to BM25; dense retrieval baselines missing

(Long) Context - Before

- separate memory, learned addressing, read/write operations
 - Memory Networks, Weston et al, 2014
- key by penultimate hidden states
 - kNN-LM, Khandelwal, 2020 (ICLR)
- Fixed-size, differential hidden-space memory pool attached to Transformer
 - Memformer: A Memory-Augmented Transformer for Sequence Modeling, Wu et al, 2022
- Storing per-layer KV vectors from fwd pass as memory representation; replace oldest updates
 - LongMem, Wang et al, 2023 (NeurIPS)

Memory Networks, Weston et al, 2014



- Foundation of external memory: Separate memory, learned addressing, read/write operations
- Stores memory as raw objects. Learnable addressing, but not representations
- Components: IGOR
- Proof of concept: outperformed LSTMs on QA requiring multi-hop inference (99% >> 29%)

kNN-LM, Khandelwal, 2020 (ICLR)

- key by penultimate hidden states
- compute input embedding $\rightarrow$ kNN retrieval similar contexts from datastore during inference $\rightarrow$ inverse distance next token distribution
$$P(y|x) = (1-\lambda)P_{LM}(y|x) + \lambda P_{kNN}(y|x)$$
- improved on rare pattern prediction

Memformer, Wu et al, 2022

- Fixed-size, differential hidden-space memory pool attached to Transformer
- external memory slots
 - recurrently write w attention
 - forgetting, exponential decay
- process sequences in segments
 - current presentation + memory
- Memory Replay Back-Propagation

LongMem, Wang et al, 2023 (NeurIPS)



- Storing per-layer KV vectors from fwd pass as memory representation
- rolling-window oldest updates
- Proof of concept on full-book language modeling tasks (context >> fixed window)

End-to-End Memory Models¹: The Case for an Explicit, Addressable, and Differentiable Memory Doctrine

- In a pre-transformer world, the prevailing idea was that latent hidden states were an unstable substrate for long-term dependencies and multi-step reasoning
- Thus, E2E proposed **explicit, addressable, and differentiable memory** s.t. memory access was embedded into neural processing and accessed as “hops”
- The core relevance to MemoryLLM² is the architectural doctrine that **memory should exist as an explicit, addressable and differentiable component inside the network**

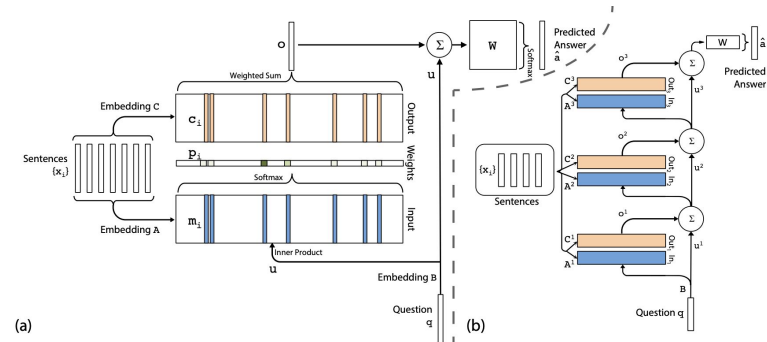


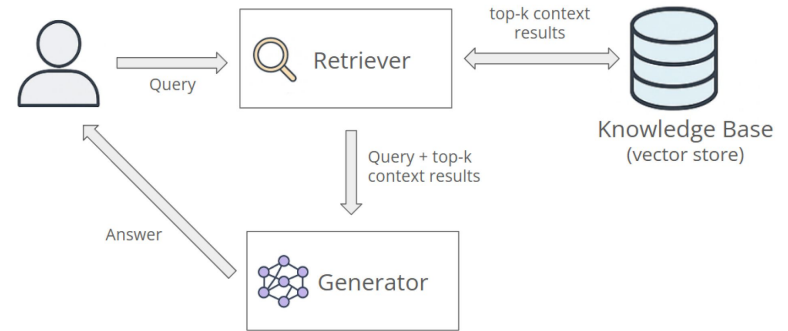
Figure 1: (a): A single layer version of our model. (b): A three layer version of our model. In practice, we can constrain several of the embedding matrices to be the same (see Section 2.2).

¹arxiv.org/abs/1503.08895.

²E2E explicitly cited in MemoryLLM as “shar[ing] certain similarities”.

Transformers: Attention as the Illusion of Holistic Memory

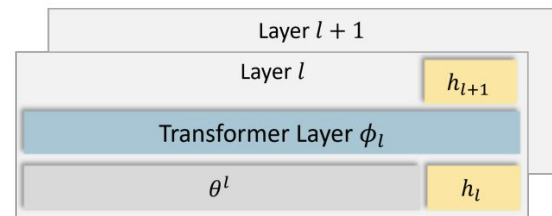
- Transformers centered memory into static pre-trained parameters and finite attention over context, reducing the perceived need for explicit memory modules
- Prompting introduces dynamism (and is arguably CL-compatible), but **context length is finite**, bounding memory within attention
- External memory systems like RAG attempt to overcome storage limits, but **attention can only operate over context**, fundamentally constraining neural processing



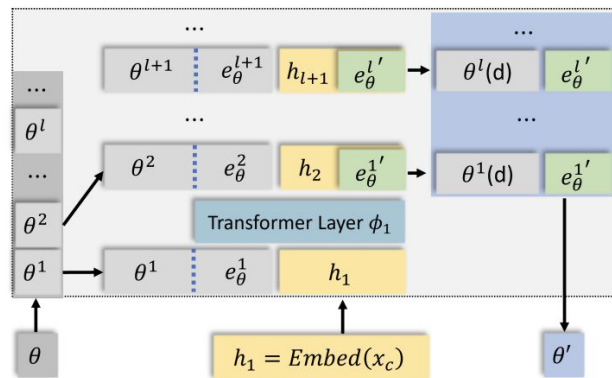
¹[Image Credit.](#)

MemoryLLM: Revival of the E2E Doctrine in Transformers

- Transformers failed to solve updating or long-term retention, architecturally confining dynamic memory to context
- MemoryLLM revives the idea of **explicit, addressable, and differentiable memory** by embedding a dedicated memory pool available at inference time
- In this sense, MemoryLLM is the **LLM-era reinterpretation of E2E** with the same doctrine of explicit memory, now embedded in the transformer's architecture

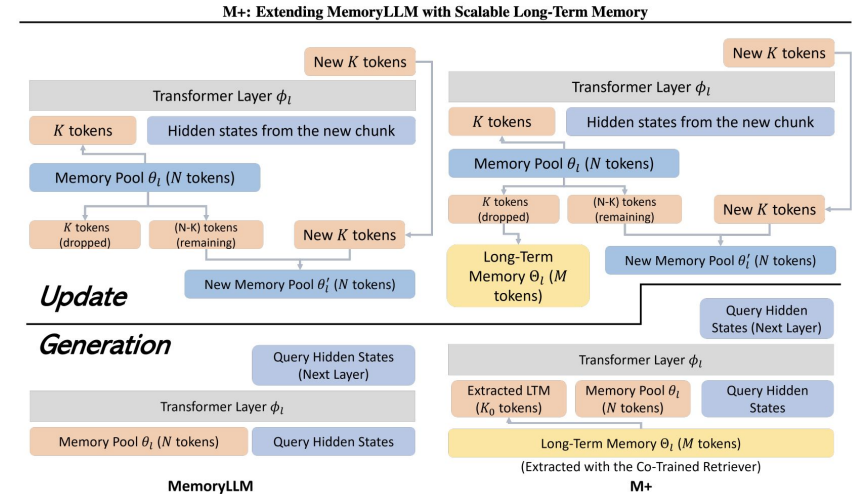


(a) Generation



M+¹: A Hybrid Doctrine

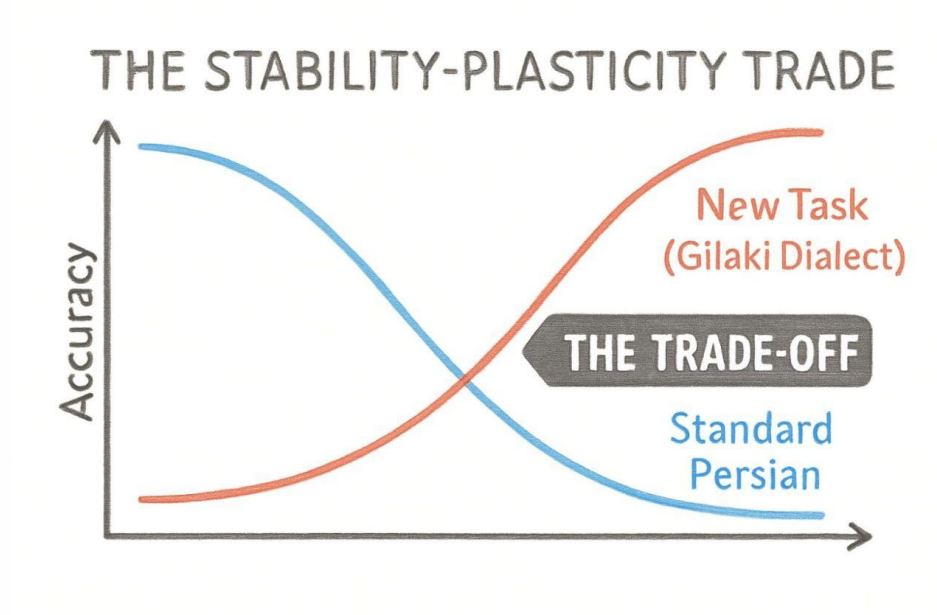
- Embedded memory ultimately suffers from a **retention horizon** that potentially enables progressive decay
- Thus, M+ categorizes memory into **short-term embedded (MemoryLLM²)** and **long-term external**
- The hybrid turn in M+ implies that **neither internal nor external memory alone yet constitutes a complete solution**



¹arxiv.org/abs/2502.00592.

²M+, unsurprisingly, explicitly cites MemoryLLM, even in the title.

Context & The Stability-Plasticity Wall

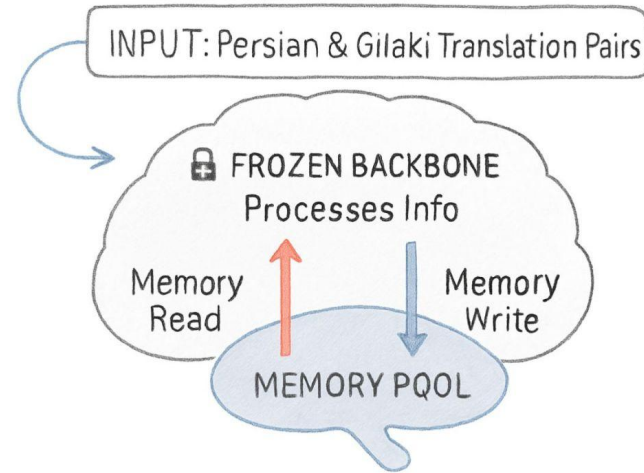


The Proposal: Project Anahita

- MemoryLLM experiments done on factual datasets:
 - zsRE
 - CounterFact
 - longBench
- Anahita will experiment on syntax/grammar

Dialect adaptation:

Optimization Problem (fighting gradients) $\Rightarrow$ Composition Problem (mixing frozen weights with active memory)



RESEARCH QUESTION: Can it store abstract RULES, not facts?

Broader Applications and Future Directions

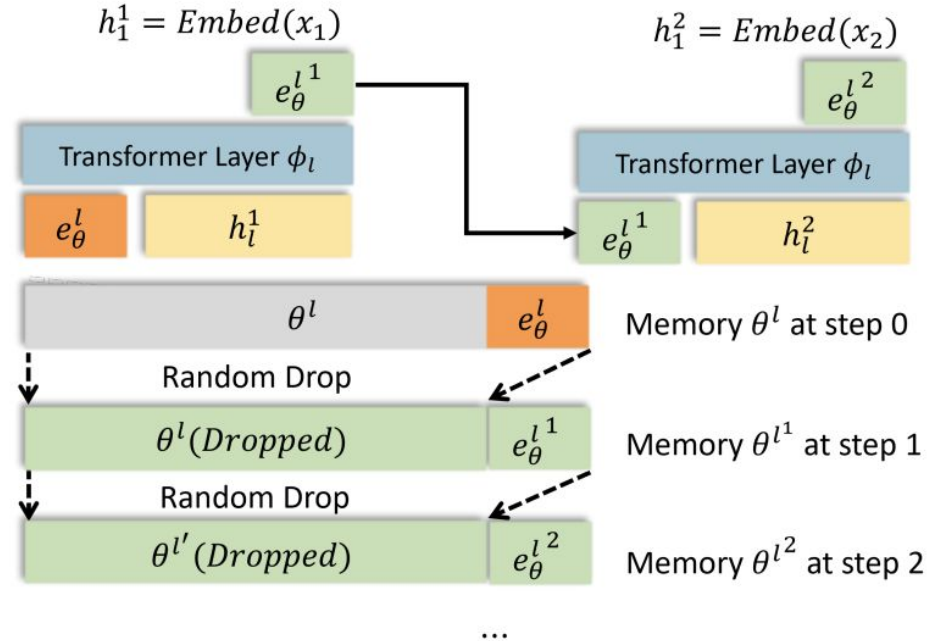


Better Eviction Mechanisms

What is memory?

Brain parts	Memory processes
Frontal lobe	Working and STM
Temporal lobe	Hearing sense
Parietal lobe	Spatial memory
Occipital lobe (visual cortex)	Visual memory
Thalamus	Attention
Mamillary body	Episodic memory
Caudate nucleus	Memories of instinctive skills
Caudate nucleus (temporal, right)	Instinctive actions, e.g., grooming is stored here
Putamen	Procedural skills
Putamen (temporal, right)	Learning skills, e.g., the ability to ride a motorbike
Amygdala	Emotional memories
Hippocampus	Activated to transfer experiences into memories, e.g., STM to LTM
Cerebellum	Conditional memories, e.g., events linked by time
Central executive (frontal, right)	Holds entire plan, including visual elements

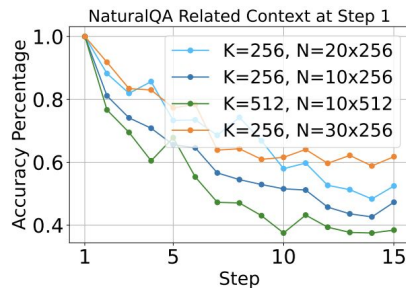
STM - short-term memory, LTM - long-term memory



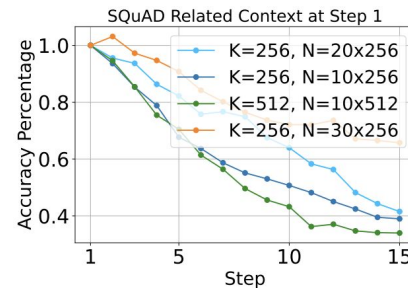
Importance of proper eviction

Retention Rate

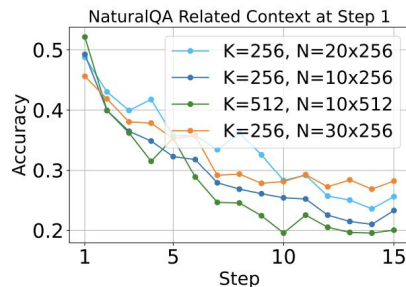
$$\left(1 - \frac{K}{N}\right)^{N/K}.$$



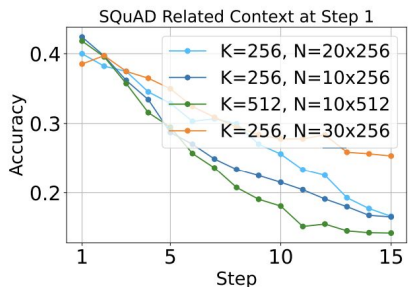
(a) NaturalQA Acc Percentage



(b) SQuAD Acc Percentage

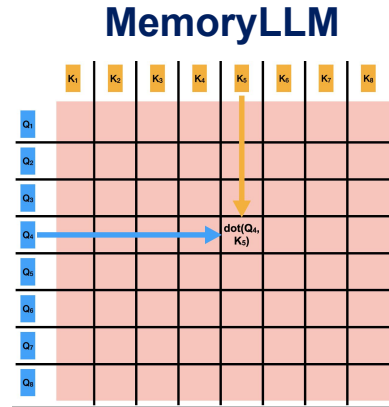


(c) NaturalQA Accuracy



(d) SQuAD Accuracy

Novelty



M+

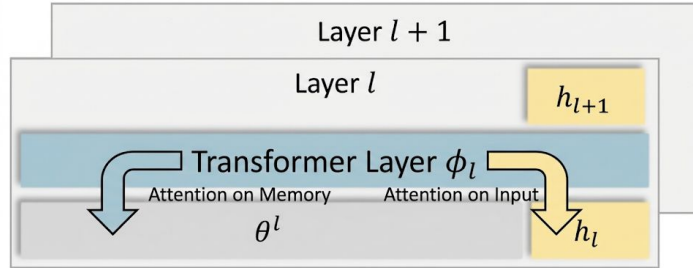


What about other methods?

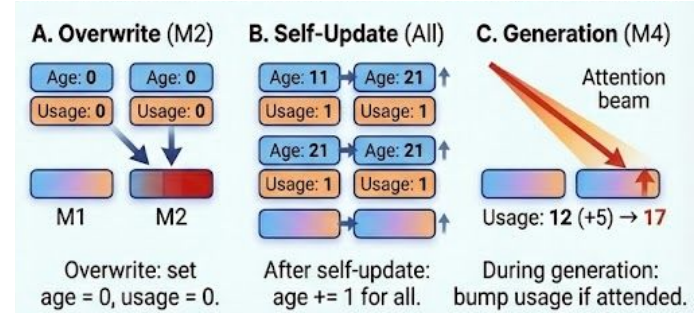
- Age + usage heuristics
- Attention-based scoring
- Scorer network?

Proposed Experiments

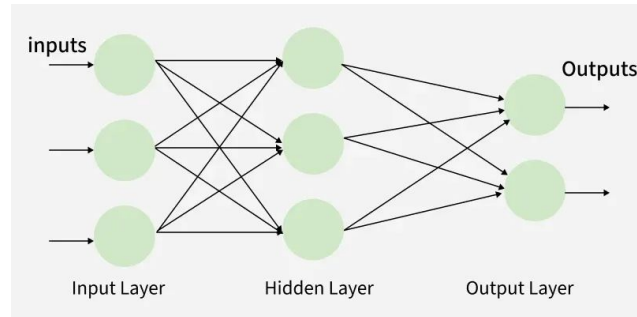
Attention-based Scoring



Age + Usage-based Scoring



Scorer Network

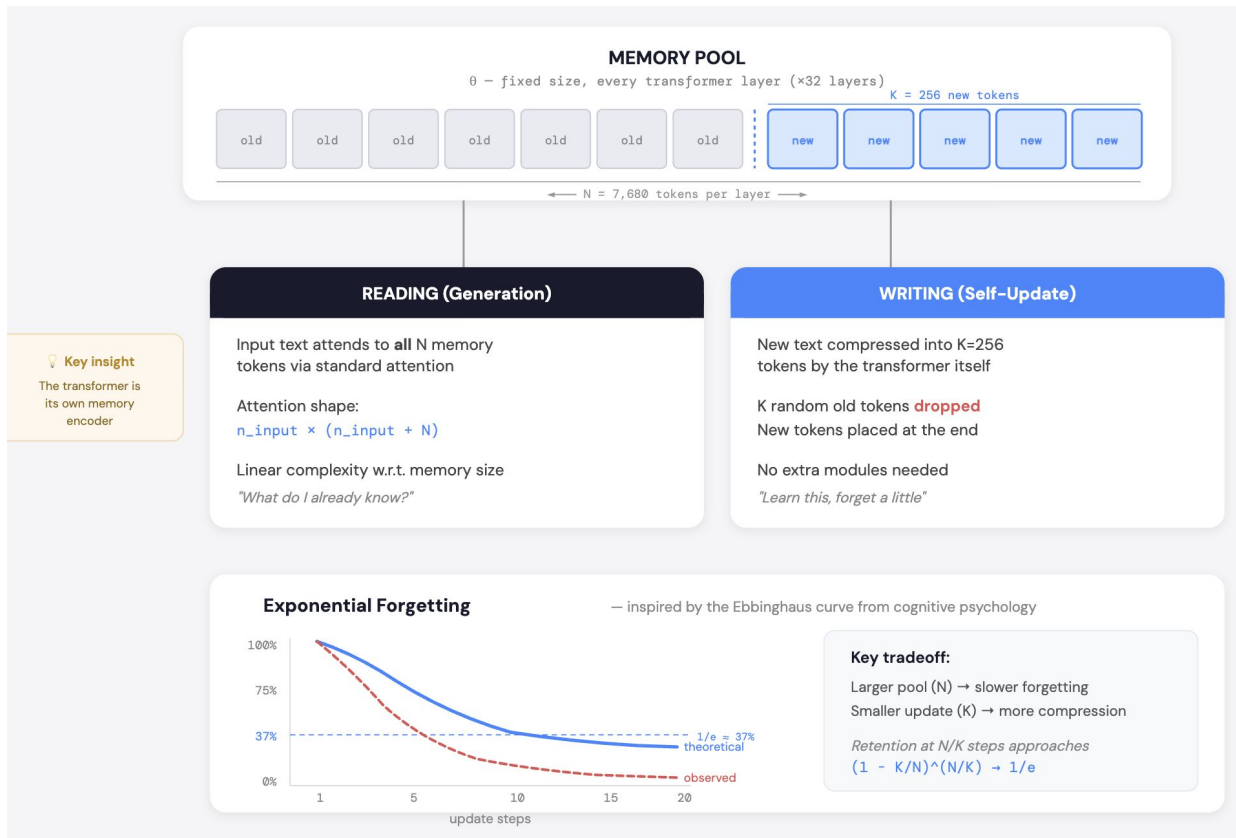


Why Should You Care?

- AI models learn everything during training, **then stop learning**
 - Like graduating college and never reading another book
- Updating them today means retraining (expensive) or bolting on external tools

We ask: *what if the model could read and remember on its own?*

How it Works: Visualization



What it Achieves Honestly

- Absorbs new facts without retraining and without external databases
- Handles long documents with constant compute cost (1 GPU, any input length)
- Survives 650K+ consecutive updates without breaking
- Not a magical solution:
 - Forgets faster in practice than the math predicts
 - Only tested on one training dataset (C4)
 - Unknown how it interacts with instruction tuning / RLHF

Where this Sits in the Research Landscape

- Three ways to keep an LLM current: retrieval, editing, or memory
- MemoryLLM opens the "memory" path at meaningful scale (1B params)
- Complementary to RAG — could process retrieved docs, not replace retrieval
- Open design question: random forgetting vs. structured addressing
 - Could more intelligent write/read mechanisms (like neural Turing machines) do better?

The Problem

Wild Card (Science Communicator): Justin Hou

CloudCopilot

Self-Updating Memory for Enterprise Code Intelligence



Our Product:

Enterprise Code Copilot: help engineers navigate, modify, and debug large codebases efficiently.

Challenges

- Scale: Enterprise codebases contain 5M+ LoC
- Velocity: Large codebases change rapidly with 50+ PRs and 10K+ LoC modified daily

Our product needs to stay up-to-date on the latest codebase changes, while providing low-latency updates for developer efficiency.

CloudCopilot

Self-Updating Memory for Enterprise Code Intelligence



Current Approaches and Limitations

- Extremely long context windows
 - Massive token costs
 - Huge latency from $O(n^2)$ computational complexity of attention
- RAG on chunked code embeddings
 - Stale code in vector databases
 - Misses global code understanding due to chunking
- Model retraining
 - Impractical for rapid updates
 - Unsustainable costs

CloudCopilot

Self-Updating Memory for Enterprise Code Intelligence



Our Proposal

- Implement MemoryLLM (ICML 2024)
 - ***Fixed size latent memory pool integrated into the transformer layers***

Strengths

- Model learns the codebase in compressed internal representation
 - better codebase-level reasoning
- Scalability of Memory Size: efficient self-updates
 - only most recent K tokens from the memory pool as input during self-updates
- Memory replacement: integrate new knowledge by dropping random K old tokens
 - robust updates
- Exponential forgetting of old information
 - enables continuous adaptation

CloudCopilot

Self-Updating Memory for Enterprise Code Intelligence



Potential Pitfalls and Concerns

- Malicious or poor quality code changes can get updated in the latent representation of memory pool
 - Cannot be easily deleted as in RAG
 - Solution: maintain memory checkpoints, roll-back if needed
- Latent memory makes traceability and source referencing challenging
 - Solution: Hybrid architecture with RAG grounding allows for source retrieval
- Privacy and Security
 - Leaked secrets in code get integrated in latent memory
 - Solution: Pre-processing and redaction by scanning for secrets before memory update

Questions / Comments:

1. For the training process for new knowledge incorporation (Figure 2), they choose the with-gradient vs. without gradient path as 50/50. Do you think there is any sensitivity to that probability or this is just a stable default?
2. In the Random Dropping section, the authors point out that they have “experimented with aggregating existing memory and new memory instead of using random dropping.” However, they found that this hurts extractability. Why does ‘exponential by decay’ behave so differently from ‘exponential by dropping’ in terms of recoverability?

Questions / Comments:

1. The authors design both static and self-updatable memory parameters. Therefore, up to a certain point we do not need to retrain the model if we switch to a different context - it will accumulate new knowledge on its own. I am curious at what point it becomes more useful to retrain the static parameters in the case of a regime switch - how much must the distribution change?
2. In the end, the authors mention, “In addition, we aim to extend MemoryLLM to be multimodal, as the memory tokens of MemoryLLM may be suitable for storing multimodal knowledge.” I am wondering if the implied architecture for the multimodal version has separate memory tokens for each modality, or if the memory should be shared. Or could it be a hybrid - some universal memory component, but then individual parts to remember modality-specific things - to encourage cross-modality abstraction but to avoid interference?

Questions / Comments:

1. A normal transformer embeds inputs into the context ('memory') with key/value projections. How is learning to create memory tokens different than learning to create keys/values?
2. What may be some strategies to retain certain tokens longer, rather than deterministically dropping/updating them from the memory matrix?

Questions / Comments:

1. The paper argues MemoryLLM and RAG are "orthogonal", wherein "RAG is used to conduct coarse retrieval from millions of documents, while MemoryLLM can process the retrieved documents." But the BM25 hybrid shows mixed results (explicit retrieval helps on some datasets, hurts on others), why is this? One hypothesis: for multi-hop tasks, BM25 retrieves fragments independently and misses cross-document dependencies that MemoryLLM's dense attention over the full memory pool captures naturally. How would you design an experiment to test whether MemoryLLM's advantage is specifically in tasks requiring joint reasoning over multiple injected facts, rather than isolated fact lookup? Also not to mention, MemoryLLM's memory is fixed-size with exponential forgetting, while RAG indexes ideally need to scale to TB w/ lossless storage.
2. Interesting finding from Sec 4.7 (Ablations): $K=128$ collapses acc to 0.25/0.34, while $K=256$ gives 0.39/0.46. Yet the forgetting analysis (Eq. 4) pushes toward minimizing K , and the ablations confirm "with the improvement of N/K , we can enable better knowledge-retention ability." So retention wants K small, but compression quality needs K large enough. The paper also notes aggregation alternatives fail because they "disrupt both the original and new knowledge," suggesting these representations are geometrically fragile. Thus, what determines the minimum viable K , and is the collapse at $K=128$ a compression failure (can't encode the information) or a retrieval failure (it's encoded but attention can't extract it)? Since the N/K ratio governs both retention and compression quality in opposing directions, how would you characterize the optimal tradeoff, and could adaptive K (varying per context entropy) improve it?

Questions / Comments:

1. At what point would you expect the robustness of the model to breakdown - you mention you tested the model through a million update steps, have you considered at what limit the model would likely begin catastrophic forgetting of original knowledge (and practical implications)?
2. Have you considered whether ultra-long context lengths (1 million+ tokens) alleviate the need for memory pool? As in, could the overhead of maintain state/updating memory pool be a drag on performance vs. just using an even longer context window?

Questions / Comments:

1. The authors note that “as the context length grows, the performance of MemoryLLM continues to improve”, and while the general trend is toward improvement, their benchmark scores do not monotonically increase. Why might this be the case?
2. The self-update rule is rather clever, ensuring that even if all of a particular update’s memory tokens are removed, information from that update is still distributed across other memory tokens. However, memory tokens that are further away from that particular update’s tokens inherently contain less of the original information, which ultimately leads to long-term forgetting, and I wonder whether or not retention could be prolonged with some intentional memory token preservation when a cluster of tokens is close to dying out?

Questions / Comments:

1. MemoryLLM is positioned as an alternative to retrieval-based methods, but since the memory pool is fixed-size and relies on compression, how does the model decide what information is worth retaining versus forgetting, and could this lead to losing rare but important knowledge compared to retrieval systems that store everything? Could some sort of importance-weighted memory replacement outperform random dropping?
2. How does MemoryLLM ensure that repeated memory updates don't gradually distort the model's internal representations, even if short-term accuracy remains stable? Beyond accuracy, are there better metrics for evaluating long-term representational stability?

Questions / Comments:

The fixed memory size keeping only relevant information in the memory context used for inference reminds me a lot of how CPU caches work and I think this could be used in conjunction with a variable sized memory system for better performance. How might you imagine this?

The authors mention that using the BM25 retriever could enhance model performance on certain datasets; perhaps there is a way to learn which method for information retrieval would work best? It might even be possible to derive an update rule that tries to decide when the dynamically-sized database should be consulted and used to update the fixed-size memory pool.